

Problem Statement: Stepwise Execution Analysis of Sorting Algorithms I

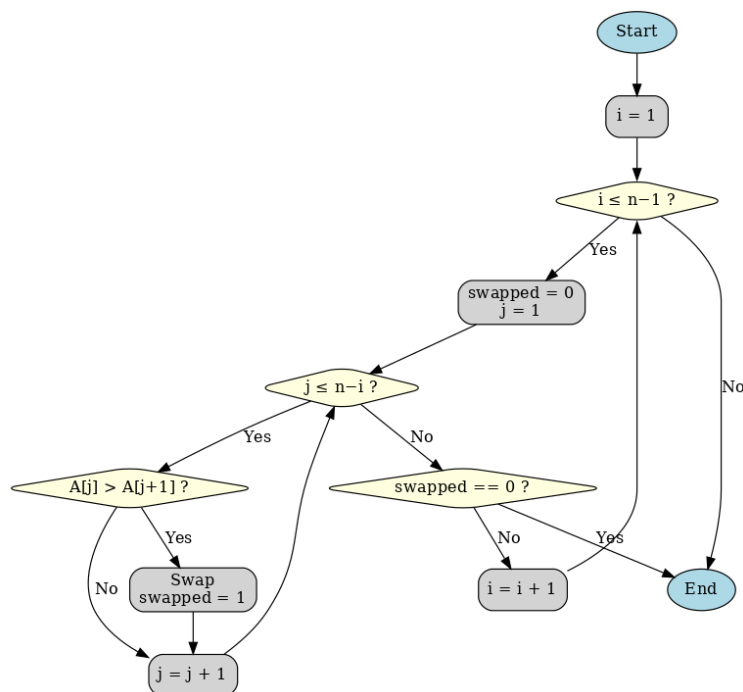
Bubble Sort

Theory:

Algorithm (Step-by-Step):

1. Repeat for $i = 1$ to $n - 1$:
2. Set swapped = false.
3. For $j = 1$ to $n - i$:
 - If $A[j] > A[j+1]$, swap them and set swapped = true.
4. After the inner loop ends:
 - If swapped == false, break (array is already sorted).
5. Otherwise, increase i and continue.
6. Finally, the array is sorted.

Bubble Sort Flowchart Layout:



Code:

```

#include <iostream>

using namespace std;

void bubbleSort(int arr[], int n) {
    bool swapped;
    int c1, c2;

    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            c1++;
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
            c2++;
        }
        if (!swapped)
            break;
    }

    cout << "Number of Comparisn: " << c1++ << endl;
    cout << "Number of Swapping: " << c2++ << endl;
}

int main() {
    int n;

    cout << "How many elements there?: ";
    cin >> n;
}

```

```

int* arr = new int[n];

cout << "Enter elements: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

bubbleSort(arr, n);

cout << "Sorted array:\n";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

Screenshots:



```

bubblesort.cpp
bubblesort.cpp > bubbleSort(int [], int)
1  #include <iostream>
2  using namespace std;
3
4  void bubbleSort(int arr[], int n) {
5      bool swapped;
6      int c1, c2;
7
8      for (int i = 0; i < n - 1; i++) {
9          swapped = false;
10         for (int j = 0; j < n - i - 1; j++) {
11             c1++;
12             if (arr[j] > arr[j + 1]) {
13                 int temp = arr[j];
14                 arr[j] = arr[j + 1];
15                 arr[j + 1] = temp;
16                 swapped = true;
17             }
18             c2++;
19         }

```

```

20         if (!swapped)
21             break;
22     }
23     cout << "Number of Comparisn: " << c1++ << endl;
24     cout << "Number of Swapping: " << c2++ << endl;
25 }
26
27 int main() {
28     int n;
29
30     cout << "How many elements there?: ";
31     cin >> n;
32
33     int* arr = new int[n];
34
35     cout << "Enter elements: ";
36     for (int i = 0; i < n; i++) {
37         cin >> arr[i];
38     }
39
40     bubbleSort(arr, n);
41
42     cout << "Sorted array:\n";
43     for (int i = 0; i < n; i++) {
44         cout << arr[i] << " ";
45     }
46     cout << endl;
47
48     return 0;
49 }
50

```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS E:\@CODES\Academic_Labs\CSE 2204 - Computer Algorithms Lab\Lab_4> g++ bubblesort.cpp -o bubblesort

>> ./bubblesort

How many elements there?: 5

Enter elements: 2 4 5 3 1

Number of Comparisn: 26

Number of Swapping: 5

Sorted array:

1 2 3 4 5
- PS E:\@CODES\Academic_Labs\CSE 2204 - Computer Algorithms Lab\Lab_4> █

Analysis:

We will trace the Bubble Sort algorithm step by step to see how it processes the input data and arrives at the final sorted array.

Input Array: $A = [2, 4, 5, 3, 1]$

Stepwise Execution (Pass by Pass):**Pass 1 ($i = 1$):**

- Compare 2 and 4 $\rightarrow$ no swap $\rightarrow [2, 4, 5, 3, 1]$
- Compare 4 and 5 $\rightarrow$ no swap $\rightarrow [2, 4, 5, 3, 1]$
- Compare 5 and 3 $\rightarrow$ swap $\rightarrow [2, 4, 3, 5, 1]$
- Compare 5 and 1 $\rightarrow$ swap $\rightarrow [2, 4, 3, 1, 5]$

Pass 2 ($i = 2$):

- Compare 2 and 4 $\rightarrow$ no swap $\rightarrow [2, 4, 3, 1, 5]$
- Compare 4 and 3 $\rightarrow$ swap $\rightarrow [2, 3, 4, 1, 5]$
- Compare 4 and 1 $\rightarrow$ swap $\rightarrow [2, 3, 1, 4, 5]$

Pass 3 ($i = 3$):

- Compare 2 and 3 $\rightarrow$ no swap $\rightarrow [2, 3, 1, 4, 5]$
- Compare 3 and 1 $\rightarrow$ swap $\rightarrow [2, 1, 3, 4, 5]$

Pass 4 ($i = 4$):

- Compare 2 and 1 $\rightarrow$ swap $\rightarrow [1, 2, 3, 4, 5]$

Final Sorted Array: $[1, 2, 3, 4, 5]$

Performance Summary:

- Number of Comparisons = **26**
- Number of Swaps = **5**

Selection Sort

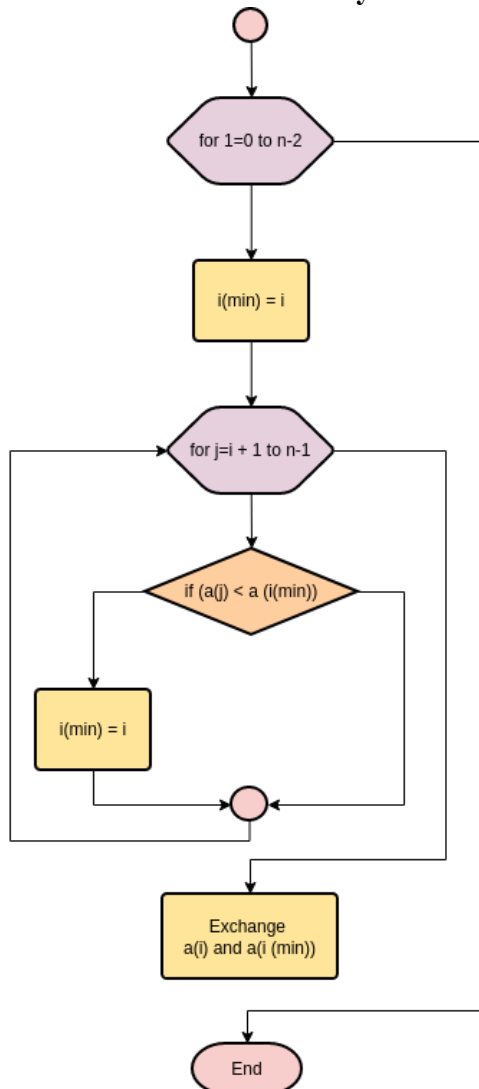
Algorithm (Step-by-Step):

Repeat for $i = 0$ to $n - 2$:

1. Set $\text{min_index} = i$.
2. For $j = i + 1$ to $n - 1$:
 - If $A[j] < A[\text{min_index}]$, then set $\text{min_index} = j$.
3. If $\text{min_index} \neq i$, then swap $A[i]$ and $A[\text{min_index}]$.
4. Increase i and continue.

Finally, the array is sorted.

Bubble Sort Flowchart Layout:



Code:

```

#include <iostream>

using namespace std;

void selectionSort(int arr[], int n) {
    int c1 = 0;
    int c2 = 0;

    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            c1++;
            if (arr[j] < arr[min]) {
                min = j;
            }
        }
        if (min != i) {
            int temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
            c2++;
        }
    }

    cout << "Number of Comparisons: " << c1 << endl;
    cout << "Number of Swaps: " << c2 << endl;
}

int main() {
    int n;

    cout << "How many elements?: ";

```

```

cin >> n;

int* arr = new int[n];

cout << "Enter elements: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

selectionSort(arr, n);

cout << "Sorted array:\n";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

delete[] arr;
return 0;
}

```

Screenshots:

selectionsort.cpp > selectionSort(int [], int)

```

1  #include <iostream>
2  using namespace std;
3
4  void selectionSort(int arr[], int n) {
5      int c1 = 0;
6      int c2 = 0;
7
8      for (int i = 0; i < n - 1; i++) {
9          int min = i;
10         for (int j = i + 1; j < n; j++) {
11             c1++;
12             if (arr[j] < arr[min]) {
13                 min = j;
14             }
15         }

```



```

16         if (min != i) {
17             int temp = arr[i];
18             arr[i] = arr[min];
19             arr[min] = temp;
20             c2++;
21         }
22     }
23
24     cout << "Number of Comparisons: " << c1 << endl;
25     cout << "Number of Swaps: " << c2 << endl;
26 }
27
28 int main() {
29     int n;
30
31     cout << "How many elements?: ";
32     cin >> n;
33
34     int* arr = new int[n];
35
36     cout << "Enter elements: ";
37     for (int i = 0; i < n; i++) {
38         cin >> arr[i];
39     }
40
41     selectionSort(arr, n);
42
43     cout << "Sorted array:\n";
44     for (int i = 0; i < n; i++) {
45         cout << arr[i] << " ";
46     }
47     cout << endl;
48
49     delete[] arr;
50     return 0;
51 }
52

```

Output:

```

PS E:\@CODES\Academic_Labs\CSE 2204 - Computer Algorithms Lab\Lab_4> g++ sele
● >> ./selectionsort
How many elements?: 5
Enter elements: 2 4 5 3 1
Number of Comparisons: 10
Number of Swaps: 4
Sorted array:
1 2 3 4 5
○ PS E:\@CODES\Academic_Labs\CSE 2204 - Computer Algorithms Lab\Lab_4> █

```

Analysis: Here, we trace the algorithm line by line to see how it processes data and arrives at the result.

Input Array: $A = [2, 4, 5, 3, 1]$

Stepwise Execution (Pass by Pass):

Pass 1 ($i = 0$):

- Compare 2 and 4 $\rightarrow$ no change $\rightarrow [2, 4, 5, 3, 1]$
- Compare 2 and 5 $\rightarrow$ no change $\rightarrow [2, 4, 5, 3, 1]$
- Compare 2 and 3 $\rightarrow$ no change $\rightarrow [2, 4, 5, 3, 1]$
- Compare 2 and 1 $\rightarrow$ min = 1 $\rightarrow$ swap $\rightarrow [1, 4, 5, 3, 2]$

Pass 2 ($i = 1$):

- Compare 4 and 5 $\rightarrow$ no change $\rightarrow [1, 4, 5, 3, 2]$
- Compare 4 and 3 $\rightarrow$ min = 3 $\rightarrow [1, 4, 5, 3, 2]$
- Compare 3 and 2 $\rightarrow$ min = 2 $\rightarrow$ swap $\rightarrow [1, 2, 5, 3, 4]$

Pass 3 ($i = 2$):

- Compare 5 and 3 $\rightarrow$ min = 3 $\rightarrow [1, 2, 5, 3, 4]$
- Compare 3 and 4 $\rightarrow$ min = 3 $\rightarrow$ swap $\rightarrow [1, 2, 3, 5, 4]$

Pass 4 ($i = 3$):

- Compare 5 and 4 $\rightarrow$ min = 4 $\rightarrow$ swap $\rightarrow [1, 2, 3, 4, 5]$

Final Sorted Array: $[1, 2, 3, 4, 5]$

Performance Summary:

- Number of Comparisons = 10
- Number of Swaps = 4

Observation Table

Test Case No.	Input Size (n)	Number of Comparison		Number of Swap/Shift		Execution Time	
		Bubble Sort	Selection Sort	Bubble Sort	Selection Sort	Bubble Sort	Selection Sort
01	10 (Best)	17	45	0	0	3.627 s	8.716 s
02	10 (Average)	43	45	13	7	2.602 s	2.713 s
03	10 (Worst)	53	45	45	5	2.758 s	2.142 s
04	100 (Best)	107	4950	0	0	3.391 s	3.671 s
05	100 (Average)	4439	4950	2091	93	3.062 s	2.885 s
06	100 (Worst)	4958	4950	4950	50	13.606 s	3.149 s
07	1000 (Best)						
08	1000 (Average)						
09	1000 (Worst)						

Conclusion: Bubble Sort, Selection Sort, and Insertion Sort are simple sorting algorithms that are best suited for small datasets or for teaching and learning purposes due to their straightforward implementation. Bubble Sort is more efficient when the dataset is already nearly sorted, as it can terminate early if no swaps are needed. Selection Sort, while making fewer swaps, always performs a fixed number of comparisons, making it more predictable but less efficient on large datasets. Insertion Sort (though not analyzed here in detail) generally outperforms both Bubble and Selection Sort for small or partially sorted data, as it requires fewer shifts in such cases. However, for larger datasets, more advanced algorithms like Merge Sort, Quick Sort, or Heap Sort are preferred because of their significantly better time complexity.